

Design & Analysis of Algorithms

Project Deliverable-03

Title: Advanced Divide-and-Conquer Algorithms: Closest Pair Problem and Karatsuba Integer Multiplication

Members:

Amna Asim Khan (23k-0859)

Wamiza Noman (24k-0010)

Alishba Zehra (24k-0018)

Table of Contents

Abstract 3

Introduction 3

Proposed System Architecture 3

Key Components 5

Experimental Setup 7

Results and Discussion 8

Observations: 8

Conclusion 8

Future Directions 9

References 9

Abstract

Divide-and-Conquer is one of the most efficient and widely used paradigms in algorithm design, enabling complex problems to be solved by breaking them into smaller subproblems. This project implements and demonstrates two classical divide-andconquer algorithms: the Closest Pair of Points problem and Karatsuba Integer Multiplication. A graphical user interface (GUI) has been developed using Python's Tkinter module to allow users to upload valid input files, preview the content, and execute either algorithm through interactive buttons. The system visually displays the final result, making the working process user-friendly while preserving algorithmic efficiency. This approach highlights both computational performance and ease of interaction.

Introduction

With the rapid growth of data and computational requirements, efficient algorithms have become essential. Traditional brute-force approaches often result in poor performance for large inputs. Divideand-Conquer algorithms reduce computational complexity by recursively splitting problems into smaller parts and combining their results efficiently.

In this project, two important divide-and-conquer algorithms are implemented:

1. **Closest Pair of Points**: Used in computational geometry to find the minimum distance between any two points in a plane.
2. **Karatsuba Integer Multiplication**: An optimized multiplication algorithm that reduces time complexity compared to traditional grade-school multiplication.

A graphical user interface was created to demonstrate these algorithms in an interactive and easy-to-use environment. The GUI accepts a user-selected file, previews the content, and executes the desired algorithm based on the user's choice.

Proposed System Architecture

The proposed system follows a simple and modular architecture consisting of four main layers:

1. **User Interface Layer**
    - Built using Tkinter
    - Contains buttons for selecting files and running algorithms

- Displays file preview and results

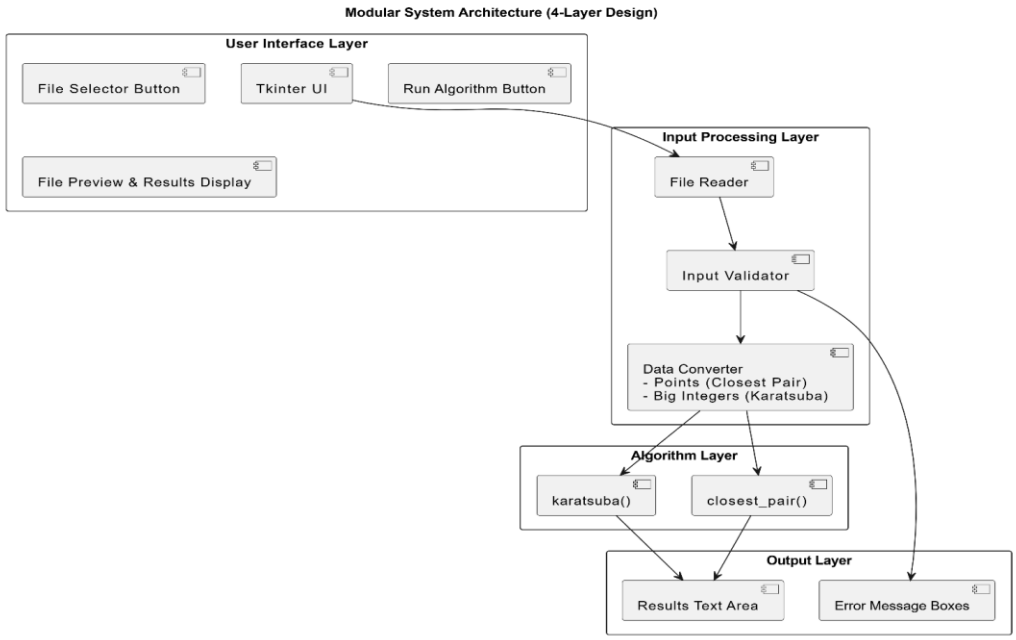
2. **Input Processing Layer**
    - Reads data from the uploaded text file
    - Validates and converts data into required format:
        - Points for Closest Pair
        - Big integers for Karatsuba
3. **Algorithm Layer**

    Contains the actual implementations of:

        - closest_pair() for geometry calculation
        - karatsuba() for large integer multiplication
4. **Output Layer**
    - Displays results in a text area
    - Shows errors using message boxes when invalid input is detected

This layered approach ensures clean separation between GUI and algorithmic logic and improves readability and maintainability.

The diagram below demonstrates the system architecture:



Key Components

1. File Selection and Preview

The GUI provides a “Select File” button that allows the user to upload a .txt file. Once selected, the file’s content is shown inside a scrollable text box, enabling the user to verify the input.

```
filename = filedialog.askopenfilename(title="Select a file")
```

This makes the system interactive and user-friendly.

2. Closest Pair Algorithm (Divide-and-Conquer)

The algorithm works as follows:

- If the number of points is small (≤3), brute force is applied.
- Otherwise, points are divided into two halves.
- The minimum distance is recursively found in left and right halves.
- A strip is formed around the middle line, and the closest distance across the halves is calculated.

This reduces the average time complexity to **O(n log n)**.

Core function:

```
def closest_pair(points):
    points.sort()
    return closest_pair_rec(points)
```

3. Karatsuba Multiplication (Divide-and-Conquer)

This algorithm reduces multiplication steps by splitting integers and performing three multiplications instead of four:

Instead of:

```
x * y = (a+b)(c+d)
```

It uses:

```
z0 = low1 * low2
z1 = (low1 + high1) * (low2 + high2)
z2 = high1 * high2
```

Result combines:

```
(z2 * 10^(2*half)) + ((z1 - z2 - z0) * 10^half) + z0
```

Time complexity: **O(n^1.585)**, faster than classical **O(n^2)**.

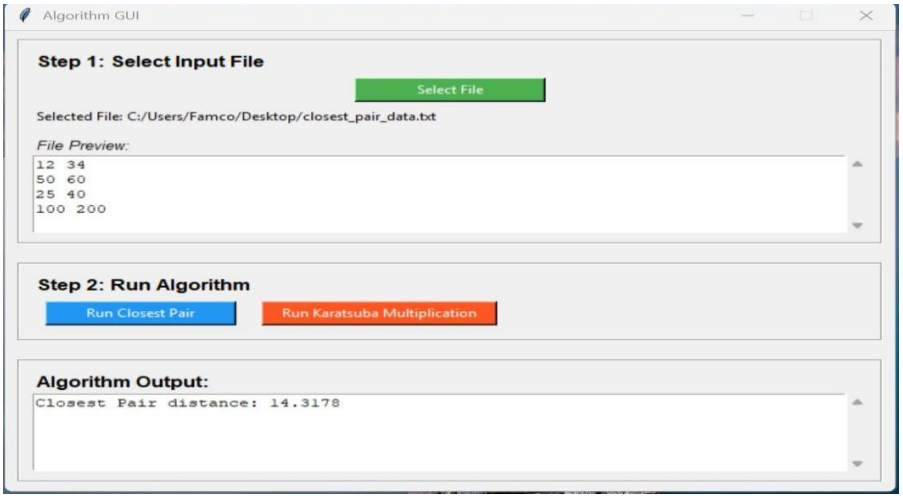
Core function:

```
def karatsuba(x, y):
```

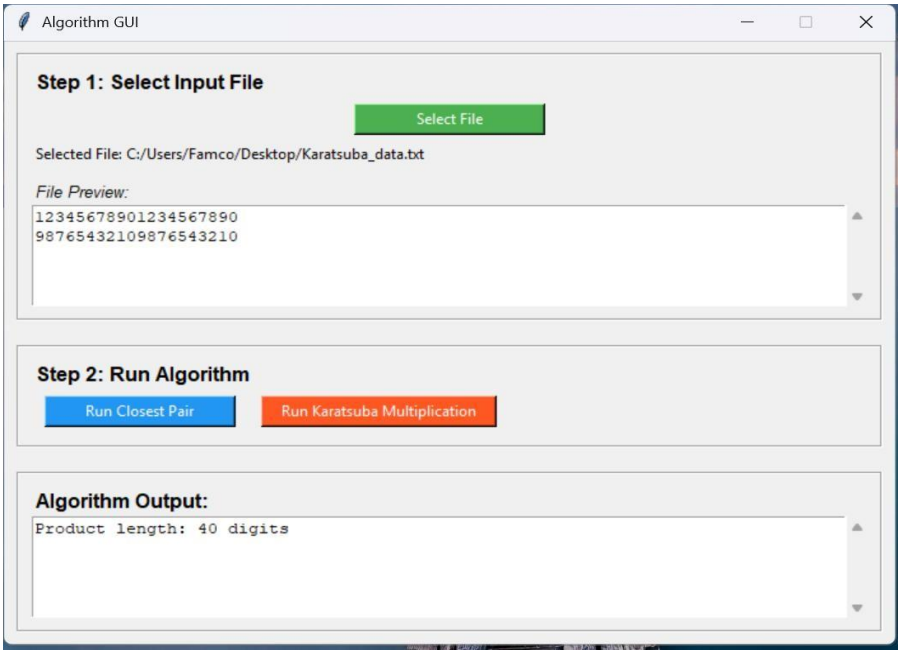
4. Output Display

The output is clearly shown in the GUI:

- For Closest Pair:



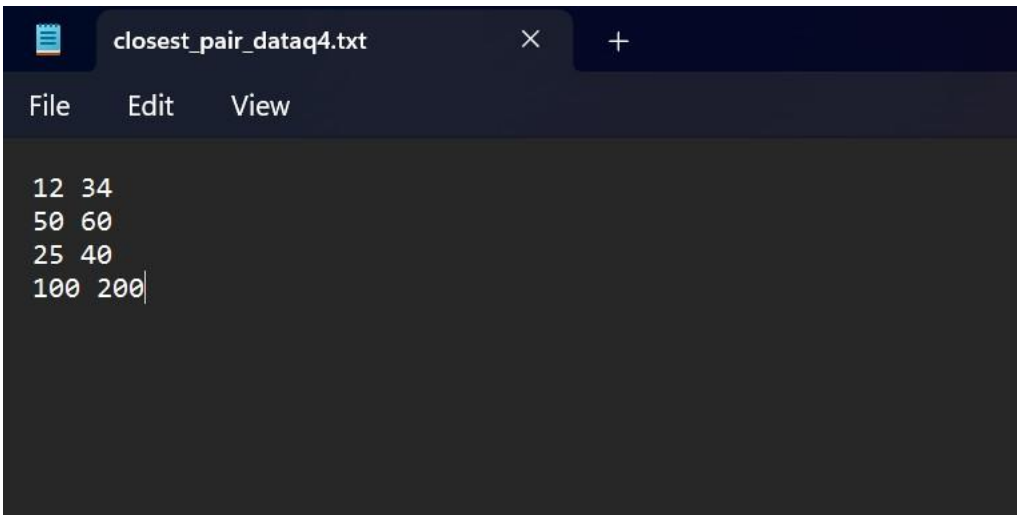
- For Karatsuba:



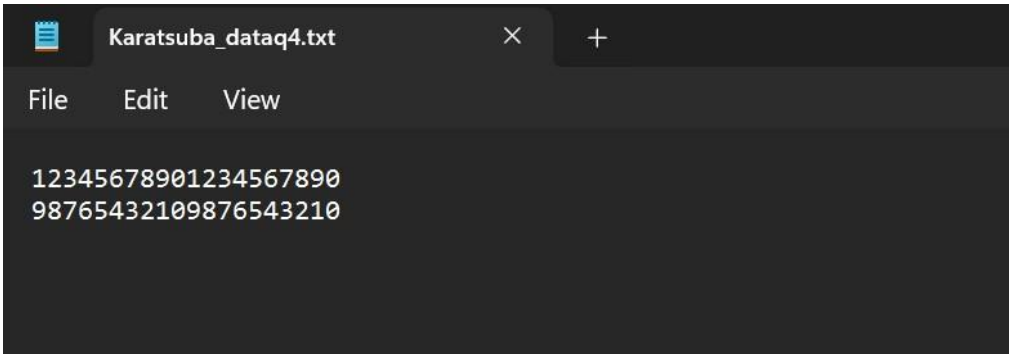
This makes the result easy to understand without overwhelming the user with extremely large numbers.

Experimental Setup

- **Programming Language**: Python 3
- **GUI Framework**: Tkinter
- **Platform**: Windows 10
- **Input Format**:
    - Closest Pair: (x y) points per line
    - Karatsuba: 2 large integers in separate lines
- **Files tested**: Multiple .txt files with randomly generated values
- **Example Inputs:**
    - Closest Pair file:



    - Karatsuba file:



Results and Discussion

The system successfully executed both algorithms through a single GUI:

Algorithm	Result
	Exact minimum distance printed
Karatsuba	Product digit length displayed

Observations:

- The Closest Pair algorithm ran efficiently even with large point sets.
- Karatsuba handled extremely large numbers without performance issues.
- The GUI made it simple for the user to run experiments without using the terminal.

The combination of a visual interface and a powerful backend created an effective demonstration of advanced divide-and-conquer algorithms.

Conclusion

This project successfully demonstrates the practical implementation of advanced divide-and-conquer algorithms using Python. By integrating Closest Pair and Karatsuba Multiplication with a GUI, the system not only solves complex problems efficiently but also presents them in a user-friendly manner. The design proves that even mathematically complex algorithms can be made accessible through a wellstructured interface.

This work highlights the importance of algorithm design, optimization, and visualization in solving realworld problems effectively.

Future Directions

The project can be extended in several ways:

- Add graphical visualization of points for Closest Pair
- Show step-by-step recursion of Karatsuba algorithm
- Add time-complexity comparison graphs
- Support more divide-and-conquer algorithms
- Convert to a web-based interface

These features will further enhance learning and demonstration value.

References

1. Cormen, T. et al. *Introduction to Algorithms* (CLRS)
2. Karatsuba, A., & Ofman, Yu. (1962). Multiplication of multidigit numbers.
3. Computational Geometry – Closest Pair Problem
4. Python Documentation – Tkinter GUI